

## 2a - C++ threads and locks

```
#include <thread>
void Foo(int a, int b) {...}
// the thread will execute foo with arguments 1, 2
std::thread t(Foo, 1, 2);
// [x, &y]: captured by value and reference;
std::thread t(x, &y) -> void [x*&y]();
t.join(); // necessary to join the thread!
// Passing a reference to std::thread:
#include <functional>; // std::ref
void ThreadBody(std::string by_value, std::string& by_reference) {...}
std::string hello = "Hello";
std::string world = "World";
// use std::ref to pass thread argument by reference
// use std::cref for const references
std::thread t(ThreadBody, hello, std::ref(world));
```

**Vector of threads:**  
std::vector<std::thread> threads;  
for (size\_t i = 0; i < 16; i++) { // launch a bunch of threads  
  threads.push\_back(std::thread(ThreadBody, a, b, c));  
  // alternative: construct an object in-place at end of vector  
  // type inference determines that std::thread constructor should be  
  invoked with ThreadBody, a, b, c  
  threads.emplace\_back(ThreadBody, a, b, c);  
} // wait for them to finish  
for (auto& thread : threads) thread.join();

**Mutex:**  
#include <mutex>  
std::mutex mutex\_x; mutex\_lock(l); mutex\_unlock(l);  
**RAII: Resource Acquisition is Initialization**  
Also known as: Constructor Acquires, Destructor Release. OR Scope-based Resource Management (Only applies to stack-allocated data)  
Widely-used pattern for resource management in C++.  
Encourages thinking about ownership and lifetimes of objects.  
If (mutex acquired when scoped, lock is constructed  
std::scoped\_lock<std::mutex> lock(mutex\_x);  
... // /mutex released when scoped\_lock goes out of scope and is  
(implicitly) destroyed

| Race conditions                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>A <b>data race</b> occurs when:<ul style="list-style-type: none"><li>• Distinct threads <b>access</b> a <b>memory location</b></li><li>• At <b>least one</b> of the accesses <b>modifies</b> the <b>location</b></li><li>• At <b>least one</b> of the accesses is <b>non-atomic</b></li><li>• The accesses are <b>not ordered</b> by synchronization</li></ul></li><li>The behaviour of C++ program P on input I is <b>undefined</b> if a data race can occur when P is executed on I (the program is allowed to do anything).</li></ul> |

**Compiler Optimization**  
The compiler optimizes your program on the **assumption that there is no undefined behaviour**. An optimization that assumes no undefined behaviour is: Sound when there is no undefined behaviour (by definition). Sound when there is undefined behaviour, because in that case **anything goes**.

```
static void Foo(int& x) { x = 1; }
static void Bar(int& x) { while (x==0) {} }
int main() { int x = 0; // then initialize and join t1 and T2 and return 0 }
Keep in a register – avoid tons of loads from memory.
x is a non-atomic variable. If another thread modified x concurrently, that would be a data race. A data race is an undefined behaviour (Assume don't happen -> no one else modifies x -> optimize)
static void Bar(int& x) { int z = 5; while (z==5) {} }
Solution: add volatile will force compiler to load x from memory each time it is accessed. (doesn't solve data race)
```

**Eliminate side-effect free infinite loop**  
• That's an **infinite loop without a side effect**  
• That's an undefined behaviour (Assume don't happen -> this code doesn't get executed -> optimize)  
static void Bar(int& x) { // Nothing }  
Solution: make x atomic so that thread must read the most up-to-date value of x.

A **data race** occurs even when both threads write the same value. E.g. When T1 and T2 both store 42 to x. (despite both threads storing 42). When T1 store 42 to x, T2 load from x, and x = 42 originally. (despite existing value of x being 42).

**std::scoped\_lock:** Constructed with one or more mutexes. Locks all mutexes on construction. Unlocks all mutexes on destruction. **Does not support:** Deferred locking - delaying locking until after construction. Early unlocking - unlocking before own destruction. Ownership transfer - using std::move to change who owns the associated mutexes.

**std::unique\_lock:** Constructed with one mutex. Locks mutex on construction (default), or locking can be deferred. Allows unlocking and relocking. Unlocks mutex on destruction if still held. Supports transfer of ownership to a distinct unique owner via std::move. Does not support management of multiple mutexes.

**std::scoped\_lock** is more **efficient**: use it unless you require the features it does not support

**Condition variables:** Use std::condition\_variable to allow threads to wait on one another.  
#include <condition\_variable>  
// Declare a condition variable; usually a class member  
std::condition\_variable condition;  
// Notify one or all threads waiting on the condition variable  
// notify\_one for 1 wait predicate; notify\_all for more predicates  
condition.notify\_one(); condition.notify\_all();

To wait on a condition variable:  
• Associate a mutex with a condition variable (Multiple conditions might share the same mutex)  
• Acquire the mutex via std::unique\_lock  
• Call wait, passing the lock and a predicate (Zero-argument boolean function; always a lambda in practice)  
std::unique\_lock<std::mutex> lock(mutex\_x);  
// wait until the predicate is true  
condition.wait(lock, [> bool { return ...; });  
// [...]: Capture relevant state to evaluate the predicate

**wait(lock, predicate):** Requires that lock is held by the thread that calls wait. Repeats the following: Return if predicate holds -> Release lock -> Block until another thread signals (via notify\_one or notify\_all)-> ...

```
1 class FIFOQueue {
2 // ...
3 void Enq(const int value) {
4   std::unique_lock<std::mutex> lock(mutex_x);
5   not_full_wait(lock, [this]() -> bool { return
6     count < elements.size(); });
7   elements[tail] = value;
8   tail = (tail + 1) % elements.size();
9   count++;
10  not_empty_notify_one();
11 }
12
13 int Deq() {
14   std::unique_lock<std::mutex> lock(mutex_x);
15   not_empty_wait(lock, [this]() -> bool { return
16     count == 0; });
17   int result = elements[head];
18   head = (head + 1) % elements.size();
19   count--;
20   not_full_notify_one();
21   return result;
22 }
```

**Reader-writer locks:** std::shared\_mutex, std::shared\_lock, from <shared\_mutex>  
**Recursive (re-entrant) mutexes:** std::recursive\_mutex, from <mutex>  
**Deferred locking:**  
// std::defer\_lock: Tag, indicating that mutex should not be locked on construction  
std::unique\_lock<std::mutex> l(m, std::defer\_lock);  
if (...) // Mutex can later be acquired if needed. RAI! still means that mutex is released on lock destruction  
l.lock();

## 2b - C++ atomics

| Atomics                                                                                                                                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Allow well-defined race conditions</li><li>• Allow fine-grained synchronization between threads</li></ul>                                                                                           |
| <b>Read-modify-write (RMW) operations</b><br>An RMW operation on an atomic variable: Reads the old value of the variable. Writes a new value to the variable. The new value is either a completely new value or a function of the old value |
| <b>Local spinning:</b> Reduce bus traffic<br>• Repeatedly load cached copy of lock state<br>• Only do TAS when the lock is seen to be available<br>TAS is still needed: another thread might get there first!                               |
| <b>Bus traffic still a problem:</b> Many threads locally spinning. Threads detect lock is free almost simultaneously. Will have a TAS fight.                                                                                                |
| <b>Active backoff:</b> On each iteration of local spinning, do nothing for a while!<br><b>NOTE:</b> use volatile for loop for this.                                                                                                         |
| <b>Passive backoff:</b> Use one of these instructions instead of volatile loop: Processor instructions for hinting that program is spinning: x86: pause. ARM: WFE.                                                                          |
| Allows the processor to "do nothing" more efficiently. Reduces energy consumption. Does not induce OS context switch.                                                                                                                       |
| <b>NOTE:</b> use __mm_pause() for this.                                                                                                                                                                                                     |
| • Short backoff: Little time wasted "doing nothing". Threads may attempt TAS at uncomfortably similar times.                                                                                                                                |
| • Long backoff: More chance threads will diverge and attempt TAS at different times. More time spent "doing nothing" - inefficient.                                                                                                         |
| <b>Exponential backoff:</b>                                                                                                                                                                                                                 |
| • Double the amount of time spent "do nothing" every iteration of local spinning                                                                                                                                                            |
| • Start from some small initial value                                                                                                                                                                                                       |
| • Stop doubling when a maximum value is reached                                                                                                                                                                                             |
| • Widely-used in networking to avoid network congestion when re-transmitting data                                                                                                                                                           |
| Exponential backoff is not used for generic spinlocks in practice: backoff parameters require application-specific tuning to work well.                                                                                                     |
| <b>Backoff can underutilise critical section when lock is contended</b>                                                                                                                                                                     |
| • Threads back off when they notice contention                                                                                                                                                                                              |
| • When a thread releases the lock, there may be some delay before any thread acquires it                                                                                                                                                    |
| • More pronounced 显著的 at the higher contention                                                                                                                                                                                              |

**Spinnlocks and fairness:**  
If contention is high, a thread may repeatedly fail to get the lock  
**Negative consequence of unfairness: starvation of threads** Positive consequence of unfairness:  
• Suppose lock grants access to shared data structure  
• Thread accesses data structure -> relevant data is in cache  
• Thread gets access again directly -> data still in cache  
Tension between fairness and performance

**Ticket lock**  
When a thread wants the lock, it obtains a ticket.  
A thread holds the lock when its ticket number is being served.  
When a thread releases a lock, the next ticket number gets served.

```
1 struct ticket_lock {
2   std::atomic<size_t> next_ = {0};
3   std::atomic<size_t> curr_ = {0};
4   void lock() {
5     auto ticket = next_.fetch_add(1, std::relaxed);
6     while (curr_.load(std::acquire) != ticket) {
7       __mm_pause();
8     }
9   }
10  void unlock() {
11    auto current = curr_.load(std::relaxed);
12    curr_.store(current + 1, std::release);
13  }
14};
```

**NOTE:** performs worse than spinlock, but more fair.

## Memory Ordering for RMW:

| k.fetch_add(1);            | Sequentially consistent load and store |
|----------------------------|----------------------------------------|
| k.fetch_add(1);            | Relaxed load and store                 |
| std::memory_order_relaxed; | Relaxed load, release store            |
| k.fetch_add(1);            | Relaxed load, release store            |
| std::memory_order_release; | Acquire load, relaxed store            |
| k.fetch_add(1);            | Acquire load, relaxed store            |
| std::memory_order_acquire; | Acquire load, release store            |
| k.fetch_add(1);            | Acquire load, release store            |
| std::memory_order_acq_rel; |                                        |

## 3a - Spinlocks

Useful when lock delay is expected to be short: Short critical sections and/or Low contention. Avoids making OS system calls.

## Test-and-set spinlock

**Test-and-set (TAS):** Performed atomically:

```
1 struct tas_lock {
2   std::atomic<bool> latch_ = {false};
3   void lock() { while (latch_.exchange(true,
4     std::acquire)); }
5   void unlock() { latch_.store(false,
6     std::release); }
7 };
```

Problem: **Cache thrashing:** When multiple cores spin on a shared lock variable, each core repeatedly reads the lock variable. When one core executes TAS on the lock variable, the memory location of the shared lock variable becomes invalid on other cores (i.e. this core invalidates all other cores). Creates excessive traffic on memory bus.

**Local spinning:** Reduce bus traffic

• Repeatedly load cached copy of lock state  
• Only do TAS when the lock is seen to be available  
TAS is still needed: another thread might get there first!

```
1 struct ttas_lock {
2   ...
3   void lock() {
4     while (latch_.exchange(true, std::acquire)) {
5       while (latch_.load(std::relaxed));
6     }
7   }
8   ...
9 };
```

## Simple implementation

```
1 struct mutex_simple {
2   const int kFree = 0; const int kLocked = 1;
3   std::atomic<int> state_ = {kFree};
4   void lock() {
5     while (state_.exchange(kLocked) == kLocked) {
6       mutex_wait(&state_, kLocked);
7     }
8   }
9   void unlock() {
10    state_.store(kFree);
11    mutex_wake(&state_, 1); // wake up 1 waiter
12  }
13 };
```

Pointer p passed to FUTURE\_WAIT and FUTURE\_WAKE can be address of any int-sized piece of data  
The OS doesn't need to know about this data before you call mutex  
The mutex system call doesn't care what meaning you ascribe to the values of \*p  
We store future waiter queues in kernel space, when a thread p calls FUTURE\_WAIT, we create new wait queue keyed by the address p. When all waiters are woken, queue memory is freed.

**Barging:** a sleep thread is woken, but another new thread acquire the lock before the woken thread.  
Problem: wasteful to call FUTURE\_WAKE when there are no waiters

## Smarter mutex-based mutex design

Three state values:  
• 0: mutex is available  
• 1: mutex is locked; the thread that locked it thinks there are no waiters  
• 2: mutex is locked; the thread that locked it thinks there might be waiters

```
1 class mutex_smarter {
2   const int kFree = 0;
3   const int kLockedNowWaiters = 1;
4   const int kLockedWaiters = 2;
5   void lock() {
6     int old = cmpxchg(kFree, kLockedNowWaiters);
7     if (old == kFree) return; // got lock w/out contention
8   }
9   do {
10    if (old == kLockedWaiters ||
11      cmpxchg(kLockedNowWaiters, kLockedWaiters) != kFree) {
12      mutex_wait(&state_, kLockedWaiters);
13    }
14    old = cmpxchg(kFree, kLockedWaiters);
15    while (old != kFree);
16  } while (old != kFree);
17  void unlock() {
18    if (state_.exchange(kFree) == kLockedWaiters) {
19      mutex_wake(&state_, 1);
20    }
21  }
22 private:
23   std::atomic<int> state_ = {kFree};
24   int cmpxchg(int expected, int desired) {
25     int e = expected; // mutable stack variable
26     state_.compare_exchange_strong(e, desired);
27     return e;
28 };
```

## 3b - Futexes and Hybrid Locks

**Futexes and Hybrid Locks**  
**Spinlocks vs. sleeping locks:** System calls are expensive; going to sleep is expensive. Spinning involves doing work all the time: being awake is expensive.  
**Spinlock:** Checks intensively check whether lock is free. Checks performed entirely in user space: no system calls required. Great when there is low contention, or short critical sections. **Prevents CPU from doing other useful work when contention is high.**  
**Sleeping lock:** Thread that doesn't manage to get lock asks OS to put it to sleep. **OS wakes thread up when lock becomes free: requires system calls.** Good idea to sleep if lock won't be free for a while: CPU can do other things. **Bad idea to sleep if lock is just about to become free.**  
**Futex:** fast userspace mutex: futex is a Linux system call - exposes functionality offered by the kernel (so not userspace). But futex works on userspace data, and can be used to implement synchronisation primitives that execute primarily in userspace. futex is very flexible and can implement more than just mutexes.

```
1 // 'SYS_FUTEX' is a SYSCALL with many OPERATIONS
2 int futex(uint32_t *p, int op, uint32_t v, const struct timespec *ttimeout, uint32_t v2, uint32_t v3) {
3   return syscall(SYS_futex, p, op, v, timeout, p2, v3);
4 }
```

```
5 // 'FUTEX_WAIT' operation:
6 // - IF *p != v' THEN sets *errno=EAGAIN' and returns '-1' immediately
7 // - OTHERWISE puts thread to sleep & adds it to KERNEL SPACE wait queue associated with *p; returns '0' when woken up
8 int futex_wait(uint32_t *p, uint32_t v) {
9   return futex(p, FUTEX_WAIT, v, nullptr, nullptr, 0);
10 }
11 // 'FUTEX_WAKE' operation:
12 // - Wake up at most 'wake_count' threads in 'p'-associated wait queue; returns 'actual number of woken up threads'
13 // - 'wake_count' most commonly '1' or 'INT_MAX'
14 int futex_wake(uint32_t *p, uint32_t wake_count) {
15   return futex(p, FUTEX_WAKE, wake_count, NULL, NULL, 0);
16 }
```

**Simple implementation**  
1 struct mutex\_simple {  
2 const int kFree = 0; const int kLocked = 1;  
3 std::atomic<int> state\_ = {kFree};  
4 void lock() {  
5 while (state\_.exchange(kLocked) == kLocked) {  
6 futex\_wait(&state\_, kLocked);  
7 }  
8 }  
9 void unlock() {  
10 state\_.store(kFree);  
11 futex\_wake(&state\_, 1); // wake up 1 waiter  
12 }  
13 };

Pointer p passed to FUTURE\_WAIT and FUTURE\_WAKE can be address of any int-sized piece of data  
The OS doesn't need to know about this data before you call mutex  
The mutex system call doesn't care what meaning you ascribe to the values of \*p  
We store future waiter queues in kernel space, when a thread p calls FUTURE\_WAIT, we create new wait queue keyed by the address p. When all waiters are woken, queue memory is freed.

**Barging:** a sleep thread is woken, but another new thread acquire the lock before the woken thread.  
Problem: wasteful to call FUTURE\_WAKE when there are no waiters

## Smarter mutex-based mutex design

Three state values:  
• 0: mutex is available  
• 1: mutex is locked; the thread that locked it thinks there are no waiters  
• 2: mutex is locked; the thread that locked it thinks there might be waiters

```
1 class mutex_smarter {
2   const int kFree = 0;
3   const int kLockedNowWaiters = 1;
4   const int kLockedWaiters = 2;
5   void lock() {
6     int old = cmpxchg(kFree, kLockedNowWaiters);
7     if (old == kFree) return; // got lock w/out contention
8   }
9   do {
10    if (old == kLockedWaiters ||
11      cmpxchg(kLockedNowWaiters, kLockedWaiters) != kFree) {
12      futex_wait(&state_, kLockedWaiters);
13    }
14    old = cmpxchg(kFree, kLockedWaiters);
15    while (old != kFree);
16  } while (old != kFree);
17  void unlock() {
18    if (state_.exchange(kFree) == kLockedWaiters) {
19      futex_wake(&state_, 1);
20    }
21  }
22 private:
23   std::atomic<int> state_ = {kFree};
24   int cmpxchg(int expected, int desired) {
25     int e = expected; // mutable stack variable
26     state_.compare_exchange_strong(e, desired);
27     return e;
28 };
```

• Zero kernel memory used when there are no waiters  
• Waiter queues created on demand: map from address of user-space integer to queue  
• Waiter queue disappears when last waiter is woken, reappears if there are more waiters in the future  
• The futex system call doesn't care how the userspace code treats the value of the user-space integer

## 4a - Concurrency in Haskell

### 4b - Concurrency in Haskell

Haskell's MVar is a key building block for concurrency As we shall see, it allows a mixture of: shared memory concurrency and message passing concurrency.  
**Import Control.Concurrent**  
**forkIO :: IO () -> IO ThreadId**  
Kicks off the computation asynchronously in a new thread; immediately returns the new thread's id. forkIO causes a thread to run asynchronously. When main ends, the program is terminated even if threads are still running.

MVar a - mutable variable storing a value of type a  
• MVar is either empty (holds no value) or full (holds a value of type a). Create a new MVar with a given value:  
newMVar :: a -> IO (MVar a)  
Create a new empty MVar:  
newEmptyMVar :: IO (MVar a)

Take value from an MVar:  
takeMVar :: MVar a -> IO a  
Blocks until the MVar is full, leaves the MVar empty. Yields the value that was in the MVar.  
Put value into an MVar:  
putMVar :: MVar a -> a -> IO ()  
Blocks until the MVar is empty. Leaves the MVar full, containing the given value.

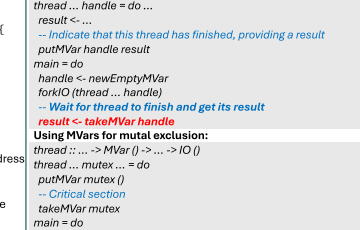
Read current value from MVar:  
readMVar :: MVar a -> IO a  
Blocks until the MVar is full. Yields the current value of the MVar, leaving it full.

**Joining threads using an MVar:**  
thread :: ... -> MVar () -> IO ()  
thread ... handle do ...  
-- indicate that this thread has finished

```
putMVar handle ()
main = do
  handle <- newEmptyMVar
  forkIO (thread ... handle)
  -- Wait for thread to finish: blocks until there is something to take
  takeMVar handle
  Getting a value from a thread join:
  thread :: ... -> MVar Int -> IO Int
  thread ... handle = do ...
    result <- ...
    -- indicate that this thread has finished, providing a result
    putMVar handle result
  main = do
    handle <- newEmptyMVar
    forkIO (thread ... handle)
    -- Wait for thread to finish and get its result
    result <- takeMVar handle
    Using MVars for mutual exclusion:
    thread :: ... -> MVar Int -> IO Int
    thread ... mutex ... = do
      putMVar mutex ...
    -- Critical section
    takeMVar mutex
    main = do
      mutex <- newEmptyMVar
      -- Launch two threads, with access to a common mutex
      forkIO (thread ... mutex ...)
      forkIO (thread ... mutex ...)
```

Replicate monad with  
replicateM :: Monad m => Int -> m a -> m [a] e.g. take from MVar 10 times in a row  
values <- replicateM 10 (takeMVar v) :: IO [Int]

### Unbounded channels



```
1 data Channel a = Channel (MVar (Stream a)) (MVar (Stream a))
2 type Stream a = MVar (Item a)
3 data Item a = Item a (Stream a)
4
5 newChannel :: IO (Channel a)
6 newChannel = do
7   emptyStream <- newEmptyMVar
8   readEnd <- newMVar emptyStream
9   writeEnd <- newMVar emptyStream
10  return (Channel readEnd writeEnd)
11 readChannel :: Channel a -> IO a
12 readChannel (Channel readEnd _) = do
13   readEndStream <- takeMVar readEnd
14   (Item value remainder) <- takeMVar readEndStream
15   putMVar readEnd remainder
16   return value
17 writeChannel :: Channel a -> a -> IO ()
18 writeChannel (Channel _ writeEnd) value = do
19   newEmptyStream <- newEmptyMVar
20   writeEndStream <- takeMVar writeEnd
21   putMVar writeEndStream (Item value newEmptyStream)
22   putMVar writeEnd newEmptyStream
```

## IORef - mutable atomic reference

```
1 newIORef :: a -> IO (IORef a)
2 readIORef :: IORef a -> IO a
3 atomically :: IORef a -> IO ()
```

### AtomicCounter

```
1 newCounter :: Int -> IO AtomicCounter
2 incrCounter :: Int -> AtomicCounter -> IO Int
3 readCounter :: AtomicCounter -> IO Int
4 decrementCounter :: AtomicCounter -> IO Int
```

### 4b - Concurrency in Rust

```
1 // can only capture 'static refs
2 let h = thread::spawn(|| -> T {...});
3 h.join().unwrap(); // remember to join handle
4
5 // attach condvar to data with this
6 let p = Arc::new(Mutex<T>, Condvar)> = ...;
7 let m (cv, cv) = &p; // unpack mutex & condvar
8 cv.notify_one(all()); // condvar notify
9 let mut guard = m.lock().unwrap();
10 let val = &mut *guard; // ASIDE: mut deref
11 while /doctual cond -> / { // wait for condvar
12   guard = cv.wait(guard).unwrap();
13 }
```

### 5 - Dynamic Data Race Detection

#### Vector clock-based race detection

A thread's **logical clock**, a non-negative integer. A thread's logical clock value **increases** each time the thread releases a mutex. Releasing a mutex indicates transitioning to next phase of concurrent program.

A **vector clock** is a tuple of N logical clocks, one per thread:  
(c<sub>0</sub>, c<sub>1</sub>, ..., c<sub>N-1</sub>). Equivalently: vector clock is a mapping from Threads to natural numbers. If V is a vector clock, V(t) denotes logical clock of thread t. Set of all vector clocks: VC

**Bottom vector clock:** L = (0, 0, ..., 0). Every thread has logical clock 0.

**Partial order on vector clocks:** V<sub>1</sub> ≤ V<sub>2</sub> if and only if ∀t. V<sub>1</sub>(t) ≤ V<sub>2</sub>(t). One vector clock is "less-than or equal to" another vector clock if their logical clocks are pointwise less-than or equal to.

**Join of vector clocks:** V<sub>1</sub> ∪ V<sub>2</sub> = pointwise maximum of V<sub>1</sub>, V<sub>2</sub>  
**Increment:** inc<sub>V</sub>(V) = V[t ← V(t) + 1]. Same as V, but logical clock of t is incremented.

**Algorithm state:** (C, L, R, W):

• C : Threads -> VC. C<sub>t</sub> = C(t). C represents what thread t knows about the logical clocks of all threads. If I am thread t, then: C<sub>t</sub>(t) is my logical clock. It will always be **positive**. For u ≠ t, C<sub>t</sub>(u) = z means "I know that thread u's logical clock is at least z". When t **acquires a lock**, t gets information about the logical clocks of threads who previously held the lock.

• L : Locks -> VC. L<sub>m</sub> = L(m). L<sub>m</sub> represents the value of the vector clock associated with the last thread that released m, when the release occurred. For a lock m and thread t: L<sub>m</sub>(t) = z means that the last thread to release m knew that thread t's logical clock was at least z when it released the mutex.

• R : Locations -> VC. R<sub>x</sub> = R(x). R<sub>x</sub>(t) represents the value of thread t's logical clock when it last read from x. For a location x and thread t: R<sub>x</sub>(t) = 0 means that I has never read from x. Otherwise R<sub>x</sub>(t) was t's logical clock last time read from x

• W : Locations -> VC. Similar to R.  
Initial analysis state: C = (inc<sub>0</sub>(1), inc<sub>1</sub>(1), ..., inc<sub>N-1</sub>(1)). Thread t knows its logical clock is 1 and thinks all other threads' logical clocks are 0. L = 2m. 1. No thread has unlocked any mutex. R = 2x. 1. W = 2x. 1. No thread has read from or written to any location.

**ACQUIRE:** C' = C[t -> (C<sub>t</sub> ∪ L<sub>m</sub>)]

(C, L, R, W) -> acq(t, m) (C', L, R, W)

**READ:** W<sub>x</sub> ⊆ C<sub>t</sub>

R' = R[x -> R<sub>x</sub>[t -> C<sub>t</sub>(t)]]

(C, L, R, W) -> rd(t, x) (C, L, R', W)

**WRITE:** W<sub>x</sub> ⊆ C<sub>t</sub> R<sub>x</sub> ⊆ C<sub>t</sub>

W' = W[x -> W<sub>x</sub>[t -> C<sub>t</sub>(t)]]

(C, L, R, W) -> wr(t, x) (C, L, R, W')

**RELEASE:** L' = L[m -> C<sub>t</sub>]

C' = C[t -> inc<sub>t</sub>(C<sub>t</sub>)]

(C, L, R, W) -> rel(t, m) (C', L', R, W)

∃u. W<sub>x</sub>(u) > C<sub>t</sub>(u)

(C, L, R, W) -> rd(t, x) WriteReadRace(u, t, x)

∃u. R<sub>x</sub>(u) > C<sub>t</sub>(u)

(C, L, R, W) -> wr(t, x) ReadWriteRace(u, t, x)

## Theoretic

### Introduction

#### Formal Properties in asynchronous computation:

- **Safety:** Nothing bad happens ever. If it is violated, it is done by a finite computation.

This is a safety property as ... violates the property and can be done in finite time. The "bad thing" is ...

- **Liveness:** Something good happens eventually. Cannot be violated by a finite computation (intuition: we can always run longer and see what happens).

This can be read as "... will eventually ...". This is a liveness property as one cannot violate it in a finite number of steps: If ... yet, they may do so eventually at a later time. The "good thing" is ....

#### Amdahl's Law:

$$\text{Speedup} = \frac{1 - \text{thread execution time}}{n - \text{thread execution time}} = \frac{1}{1 - p + \frac{p}{n}}$$

$p$ : parallel fraction.  $n$ : number of threads. (We need very high parallel fraction to achieve high speedup)

#### Basics of Shared-Memory Concurrency

##### Read-Modify-Write (RMW):

- Holds some value  $x$
- Has a method `getAndMumble()` such that: Returns object's prior value  $x$ . Replaces  $x$  with `mumble(x)` for some function `mumble`. And it does so atomically, in a mutually exclusive way.

**Weak RMW:** enables synchronisation between **two** threads. (E.g. exchange – xchg, FetchAndAdd-FAA)

**Strong RMW:** enables synchronisation between an **arbitrary** number of threads. (E.g. CompareAndSet-CAS)

#### Concurrent Semantics: Sequential Consistency (SC)

**Sequential Consistency (SC)** (=interleaving semantics): The instructions of each thread are executed in order. Instructions of different threads can be interleaved arbitrarily.

##### Notations:

| $x, y, \bar{z}, \dots$ | Shared memory locations                          |
|------------------------|--------------------------------------------------|
| $a, b, \bar{c}, \dots$ | Private (thread-local) registers                 |
| $E, \bar{E}, \dots$    | Expressions over values (integers) and registers |
| $a := x$               | Read from location $x$ into register $a$         |
| $x := a$               | Write contents of register $a$ to location $x$   |
| $a := \bar{E}$         | Assignment: compute $E$ and write it to $a$      |

*CONWHILE* sequential commands:

$C \in Com ::= a := \bar{E}$  [assignment]  
[  $a := x$  [(memory) read]  
[  $x := a$  [(memory) write]  
[  $a := CAS(x, E) \} FAA(x, E)$  [(memory) RMWs]  
] skip [  $C; C$  ] while  $B$  do  $C$   
] if  $B$  then  $C$  else  $C$

*CONWHILE* concurrent programs:  $P \in Prog \triangleq Tid \rightarrow Com$

For  $n$ -threaded program  $P$ :  $C_1 || C_2 || \dots || C_n$  with  $dom(P) = \{\tau_1, \dots, \tau_n\}$  and  $P(\tau_i) = C_i$  for  $i \in \{1, \dots, n\}$ .

##### SC Configurations:

- **Shared memory:**  $M \in Mem \triangleq Loc \rightarrow Val$ .  $Val$ : set of all values, including integer and Boolean values.
- **Store:**  $s \in Store \triangleq Reg \rightarrow Val$ .
- **Store Map:**  $S \in SMap \triangleq Tid \rightarrow Store$ .  $s = S(\tau)$ : the store map of  $\tau$ .  $s(b)$ : the value of register  $b$  for thread  $\tau$ .
- **SC configuration:**  $(P, S, M)$

##### SC Operational Semantics:

**Program transitions** describe the steps in program executions, e.g. how a conditional (if) statement is executed.

**Storage transitions** describe how instructions interact with the storage (memory) system, e.g. how a memory read behaves.

- If the program takes a silent transition:

$$\frac{P, S \xrightarrow{\tau \hookrightarrow_p} P', S'}{P, S, M \rightarrow P', S', M}$$

- If both the program and storage systems take the same transition, then we can combine them:

$$\frac{P, S \xrightarrow{\tau \hookrightarrow_p} P', S' \quad M \xrightarrow{\tau \hookrightarrow_m} M'}{P, S, M \rightarrow P', S', M'}$$

##### SC Transition Labels:

- **empty label**  $\varepsilon$ : a silent transition (e.g. when reducing skip;  $C$  to  $C$ );
- **read label**  $(R, x, v)$ : reading value  $v$  from memory location  $x$
- **write label**  $(W, x, v)$ : writing value  $v$  to memory location  $x$
- **successful RMW label**  $(U, x, v_w, v_r)$ : updating the value of location  $x$  to  $v_w$  when the old value of  $x$  in memory is  $v_r$
- **failed RMW label**  $(U, x, v_w, \perp)$ : a failed CAS instruction where the old value of  $x$  in memory,  $v_w$ , does not match the expected value.

**SC Sequential Transitions:**  $C, s \xrightarrow{\tau, C'} C', s'$ .

$$\frac{s(a) = v}{s(a) = v} \quad \frac{s' = s[a \mapsto v]}{s' = s[a \mapsto v]}$$

$$x := a, s \xrightarrow{(W, x, v)}_c \text{skip}, s \quad a := x, s \xrightarrow{(R, x, v)}_c \text{skip}, s'$$

$$\text{eval}(s, E) = v \quad v_w = v_w + v \quad s' = s[a \mapsto v_w]$$

$$a := FAA(x, E), s \xrightarrow{(U, x, v_w, v_r)}_c \text{skip}, s'$$

$$\text{eval}(s, E_e) = v_e \quad v_e = v_o \quad \text{eval}(s, E_r) = v_n \quad s' = s[a \mapsto 1]$$

$$a := CAS(x, E_e, E_n), s \xrightarrow{(U, x, v_w, v_r)}_c \text{skip}, s'$$

$$\text{eval}(s, E_e) = v_e \quad v_e \neq v_o \quad s' = s[a \mapsto 0]$$

$$a := CAS(x, E_e, E_n), s \xrightarrow{(U, x, v_w, \perp)}_c \text{skip}, s'$$

##### Program transition:

$$P(\tau) = C \quad S(\tau) = s \quad C, s \xrightarrow{\tau}_c C', s' \quad P' = P[\tau \mapsto C'] \quad S' = S[\tau \mapsto s']$$

$$P, S \xrightarrow{\tau \hookrightarrow_p} P', S'$$

#### SC Storage Transitions: $M \xrightarrow{\tau, M'} M'$

$$\frac{M(x) = v \quad M' = M[x \mapsto v] \quad M(x) = v_o}{M \xrightarrow{\tau(R, x, v)}_m M \quad M' = M' \quad M \xrightarrow{\tau(U, x, v)}_m M' \quad M \xrightarrow{\tau(W, x, v)}_m M' \quad M \xrightarrow{\tau(U, x, v_o, \perp)}_m M'}$$

**Many Steps of Evaluation:**  $\rightarrow^*$  for the reflexive transitive closure of  $\rightarrow$ :  
 $P, S, M \rightarrow^* P', S', M' \Leftrightarrow (P, S, M) = (P', S', M') \vee \exists P'', S'', M'', P, S, M \rightarrow P'', S'', M'' \wedge P'', S'', M'' \rightarrow^* P', S', M'$ .

##### SC Traces:

**Initial Memory:**  $M_0 \triangleq \lambda x. 0$ . **Initial Store:**  $S_0 \triangleq \lambda a. 0$ . **Initial Store Map:**  $S_0 \triangleq \lambda \tau. S_0$ . **Terminated Program:**  $P_{skip} \triangleq \lambda \tau. \text{skip}$ .

**Initial SC-configuration:**  $(P, S_0, M_0)$ .

**SC-trace:** an evaluation path that starts from the initial SC-configuration of  $P$  and terminates with  $P_{skip}$ , i.e. there exists  $S$  and memory  $M$  such that:  $P, S_0, M_0 \rightarrow^* P_{skip}, S, M$ .

A pair  $(S, M)$  denotes an **SC-outcome** iff  $P, S_0, M_0 \rightarrow^* P_{skip}, S, M$ .

SC is **not deterministic nor confluent**.

##### Deterministic:

$\forall P, S, M, P_1, S_1, M_1, P_2, S_2, M_2$ .

if  $P, S, M \rightarrow P_1, S_1, M_1 \wedge P, S, M \rightarrow P_2, S_2, M_2$

then  $(P_1, S_1, M_1) = (P_2, S_2, M_2)$

##### Confluent:

$\forall P, S, M, P_1, S_1, M_1, P_2, S_2, M_2$ .

if  $P, S, M \rightarrow^* P_1, S_1, M_1 \wedge P, S, M \rightarrow^* P_2, S_2, M_2$

then  $\exists P', S', M'. P_1, S_1, M_1 \rightarrow^* P', S', M' \wedge P_2, S_2, M_2 \rightarrow^* P', S', M'$

#### Concurrent Semantics: Total Store Ordering (TSO)

##### Weak Memory Models (WMM):

- Unlike SC, the instructions of a thread may be **reordered**, i.e. they may execute **out of order**
- May observe **weak behaviours** (anomalies) not observable under SC.

##### TSO (Total Store Ordering)

- Allows the following for **adjacent** instructions (in program order):

1. **write-read reordering on different locations:** a later read on  $y$  can be reordered before an earlier write on  $x$  when  $x \neq y$ .

2. **write-read reordering on same location with write forwarding:** a later read on  $x$  can be reordered before an earlier write on  $x$  provided that the value of the write is forwarded to (inlined in) the read.

- 1 leads to **weak SB (store buffering)**; 2 alone **does not cause weak behaviours**.

- No other reordering or change is allowed

- i.e. TSO = SC + write-read reordering on different locations

+ write-to-read on same location + write forwarding

For TSO, we extend the *CONWHILE* sequential commands:

$C \in Com ::= mfence$  [memory fence]

##### TSO Configurations:

- **Buffer**, as a FIFO sequence of delayed write labels:  $b \in Buff \triangleq Seq < WLab >. WLab \triangleq \{(W, x, v) | x \in Loc \wedge v \in Val\}$ .
- **Buffer Map:**  $B \in BMap \triangleq Tid \rightarrow Buff$ .  $b = B(\tau)$ : the buffer of  $\tau$ .
- **TSO Configuration:**  $(P, S, M, B)$ .

**TSO Transition Label:** either an SC label, or a memory fence label  $MF$  for executing an *mfence*.

**TSO sequential transitions include all SC sequential transitions**, as

well as:

$$mfence, s \xrightarrow{MF}_c skip, s$$

**TSO Storage transitions:**  $M, B \xrightarrow{\tau, M'} M', B'$ .

$$B(\tau) = b \quad b' = b.(W, x, v) \quad B' = B[\tau \mapsto b']$$

$$M, B \xrightarrow{\tau(W, x, v)}_m M, B'$$

$$B(\tau) = b \quad \text{get}(M, b, x) = v$$

$$M, B \xrightarrow{\tau(R, x, v)}_m M, B$$

$$\text{get}(M, b, x) \triangleq \begin{cases} v & \text{if } \exists b_1, b_2. b = b_1.(W, x, v). b_2 \\ & \wedge \neg \exists v'. (W, x, v') \in b_2 \\ M(x) & \text{otherwise} \end{cases}$$

$$\frac{B(\tau) = \emptyset}{M, B \xrightarrow{\tau MF}_c M, B} \quad \frac{B(\tau) = \emptyset \quad M(x) = v_o}{M, B \xrightarrow{\tau(U, x, v_o, \perp)}_m M, B}$$

$$B(\tau) = \emptyset \quad M(x) = v_o \quad M' = M[x \mapsto v_n]$$

$$M, B \xrightarrow{\tau(U, x, v_o, v_n)}_m M', B$$

$$B(\tau) = (W, x, v). b \quad M' = M[x \mapsto v] \quad B' = B[\tau \mapsto b]$$

$$M, B \xrightarrow{\tau}_m M', B'$$

##### Initial Buffer Map: $B_0 \triangleq \lambda \tau. \emptyset$ . Initial TSO-configuration:

$(P, S_0, M_0, B_0)$ .

**TSO-trace:** an evaluation path that starts from the initial TSO-configuration of  $P$  and terminates with  $P_{skip}$  and empty buffers; i.e. there exists  $S$  and memory  $M$  such that:  $P, S_0, M_0, B_0 \rightarrow^* P_{skip}, S, M, B_0$ .

A pair  $(S, M)$  denotes an **TSO-outcome** iff  $P, S_0, M_0, B_0 \rightarrow^* P_{skip}, S, M, B_0$ .

TSO is **not deterministic nor confluent**.

**NOTE:** *TSO* allows for s fence (with SF label); and s fence-read reordering. The **buffer** for memory transitions can hold SF labels.

**NOTE:** *TSO* is like *TSO* but allows write-write reordering on different memory locations.

## Declarative Semantics

$$\text{typ}((R, x, v_r)) \triangleq R \quad \text{typ}((W, x, v_w)) \triangleq W \quad \text{typ}((U, x, v_r, v_w)) \triangleq U$$
$$\text{val}_r((R, x, v_r)) \triangleq v_r \quad \text{val}_r((U, x, v_r, v_w)) \triangleq v_r$$
$$\text{val}_w((W, x, v_w)) \triangleq v_w \quad \text{val}_w((U, x, v_r, v_w)) \triangleq v_w$$
$$\text{loc}((R, x, v_r)) = \text{loc}((W, x, v_w)) = \text{loc}((U, x, v_r, v_w)) \triangleq x$$
$$[A] \text{ for the identity relation on } A: [A] \triangleq \{(a, a) \mid a \in A\}$$

$r, r'$  for composition

$r^?$  for the reflexive closure of  $r$

$$\text{irreflexive}(r) \stackrel{\text{def}}{\Leftrightarrow} \neg \exists a. (a, a) \in r$$

$$\text{acyclic}(r) \stackrel{\text{def}}{\Leftrightarrow} \text{irreflexive}(r^+)$$